Studio Nx

JS 293 — 이펙트 6종

2026-10-19

JavaScript 293 — 이펙트 6종 (`audio/effects.js`)

항목	내용
파일	`studionx/frontend/static/js/audio/effects.js`
작업	S1-11a 재생 오디오 그래프: Gain, EQ 3-Band, Compressor, Limiter, Delay, Reverb를 Web Audio 노드로 구현 (FR-243, FR-247)
작성	Claude (AI), 2026-10-19, 사용자 검토
이전, 다음	이전 292 노트와 악기 층, 다음 294 이펙트 체인 관리

1. 하는 일

시드의 이펙트 6종을 Web Audio 기본 노드의 조합으로 만듭니다. 모든 이펙트는 같은 모양입니다.

```
{ input, output, update(params), dispose() }

input  ─▶ (이펙트 내부 노드들) ─▶ output
update  노브가 바뀔 때 새 값을 소리에 반영합니다
dispose 노드 연결을 끊어 정리합니다
```

이펙트	코드	노드 조합
Gain / Utility	`gain`	스테레오 게인, `StereoPannerNode`, 위상 반전은 게인을 음수로
EQ 3-Band	`eq3`	`BiquadFilter` 3개 (lowshelf, peaking, highshelf)
Compressor	`compressor`	`DynamicsCompressorNode`, 메이크업 게인
Limiter	`limiter`	`DynamicsCompressorNode`(레이시오 20, 니 0), 보정 게인
Delay	`delay`	`DelayNode`, 되먹임 게인, 톤 필터, 첫 반복 보정 딜레이
Reverb	`reverb`	프리딜레이, `ConvolverNode`(코드로 만든 임펄스), 원음과 섞기

2. 언제 쓰나

쓰는 곳	하는 일
`EffectRack` (294)	트랙에 붙은 이펙트마다 `createEffect`로 만들고, 노브가 바뀌면 `update`를 부릅니다.
WAV 출력 (S1-11b 예정)	같은 이펙트를 `OfflineAudioContext`에 만들어 렌더링합니다.

3. 코드

3.1 순수 함수

```
export function delaySeconds(p, bpm) {
  let sec;
  if (p.sync) {
    const beats = DIVISION_BEATS[p.division] != null ? DIVISION_BEATS[p.division] : 0.5;
    sec = beats * 60 / (bpm > 0 ? bpm : 120);
  } else {
```

```
    sec = Number(p.time) || 0.35;
  }
  return Math.max(0.001, Math.min(MAX_DELAY_SECONDS, sec));
}
```

```
export function makeImpulse(sampleRate, decay, damping, channel) {
  const length = Math.max(1, Math.floor(sampleRate * Math.max(0.1, decay)));
  const out = new Float32Array(length);
  let seed = 12345 + (channel || 0) * 7919;
  const coef = 1 - Math.max(0, Math.min(1, damping)) * 0.9;   // 1 = 그대로, 0.1 = 크게 둔하게
  let lp = 0;
  for (let i = 0; i < length; i++) {
    seed = (Math.imul(seed, 1664525) + 1013904223) >>> 0;
    const noise = (seed / 4294967296) * 2 - 1;
    const envelope = Math.pow(0.001, i / length);   // 끝에서 -60dB
    lp = lp + coef * (noise - lp);
    out[i] = lp * envelope;
  }
  return out;
}
```

```
export function compressorAutoMakeupDb(threshold, ratio) {
  const r = Math.max(1, ratio);
  const gainDb = Math.min(0, threshold) * (1 - 1 / r);   // 0dB 입력일 때의 감쇠(dB, 음수)
  return -0.6 * gainDb;
}
```

3.2 Gain 과 EQ

```
function createGainFx(ctx) {
  const gain = createStereoGain(ctx);
  const pan = typeof ctx.createStereoPanner === 'function' ? ctx.createStereoPanner() : null;
  if (pan) gain.connect(pan);
  return {
    input: gain,
    output: pan || gain,
    update: function (p) {
      const g = dbToGain(p.gain);
      gain.gain.value = p.phase_invert ? -g : g;
      if (pan) pan.pan.value = Math.max(-1, Math.min(1, Number(p.pan) || 0));
    },
    dispose: function () { drop(gain); if (pan) drop(pan); },
  };
}
```

```
function createEq3Fx(ctx) {
  const low = ctx.createBiquadFilter(); low.type = 'lowshelf';
  const mid = ctx.createBiquadFilter(); mid.type = 'peaking';
  const high = ctx.createBiquadFilter(); high.type = 'highshelf';
  low.connect(mid); mid.connect(high);
  return {
    input: low,
    output: high,
    update: function (p) {
```

```
      low.frequency.value = Number(p.low_freq) || 200;
      low.gain.value = Number(p.low_gain) || 0;
      mid.frequency.value = Number(p.mid_freq) || 1000;
      mid.gain.value = Number(p.mid_gain) || 0;
      mid.Q.value = Number(p.mid_q) || 0.7;
      high.frequency.value = Number(p.high_freq) || 6000;
      high.gain.value = Number(p.high_gain) || 0;
    },
    dispose: function () { drop(low); drop(mid); drop(high); },
  };
}
```

3.3 Compressor 와 Limiter

```
function createCompressorFx(ctx) {
  const comp = ctx.createDynamicsCompressor();
  const makeup = ctx.createGain();
  comp.connect(makeup);
  return {
    input: comp,
    output: makeup,
    update: function (p) {
      const threshold = p.threshold != null ? Number(p.threshold) : -20;
      const ratio = Math.max(1, Math.min(20, Number(p.ratio) || 4));
      comp.threshold.value = threshold;
      comp.ratio.value = ratio;
      comp.attack.value = Math.max(0, Math.min(1, Number(p.attack) || 0.01));
      comp.release.value = Math.max(0, Math.min(1, Number(p.release) || 0.2));
      comp.knee.value = Math.max(0, Math.min(40, p.knee != null ? Number(p.knee) : 10));
      // 브라우저가 자동으로 더하는 메이크업을 덜어 내고, 사용자가 정한 메이크업만 더함
      makeup.gain.value = dbToGain((Number(p.makeup) || 0) - compressorAutoMakeupDb(threshold, ratio));
    },
    dispose: function () { drop(comp); drop(makeup); },
  };
}
```

```
function createLimiterFx(ctx) {
  const comp = ctx.createDynamicsCompressor();
  const trim = ctx.createGain();
  comp.ratio.value = 20;
  comp.knee.value = 0;
  comp.attack.value = 0.001;
  comp.connect(trim);
  return {
    input: comp,
    output: trim,
    update: function (p) {
      const ceiling = Math.max(-100, Math.min(0, p.ceiling != null ? Number(p.ceiling) : -0.3));
      comp.threshold.value = ceiling;
      comp.release.value = Math.max(0.01, Math.min(1, Number(p.release) || 0.1));
      trim.gain.value = dbToGain(-compressorAutoMakeupDb(ceiling, 20));   // 브라우저의 자동 메이크업을
덜어 내야 천장을 넘지 않음
    },
    dispose: function () { drop(comp); drop(trim); },
```

```
  };
}
```

3.4 Delay

```
function createDelayFx(ctx, getBpm) {
  const input = ctx.createGain();
  const output = ctx.createGain();
  const delay = ctx.createDelay(MAX_DELAY_SECONDS);
  const tone = ctx.createBiquadFilter(); tone.type = 'lowpass';
  const feedback = ctx.createGain();
  const tap = ctx.createDelay(0.1);      // 고리를 한 바퀴 돌 때 더 걸리는 128 샘플만큼 첫 반복을 늦춤 (아래 설명)
  const wet = ctx.createGain();
  input.connect(output);                 // 원음
  input.connect(delay);
  delay.connect(tone);
  tone.connect(feedback);
  feedback.connect(delay);               // 되먹임
  tone.connect(tap);
  tap.connect(wet);
  wet.connect(output);
  // 브라우저는 되먹임 고리를 한 바퀴 돌 때마다 128 샘플(한 번 처리 단위)을 더 걸리게 합니다.
  // 그래서 고리의 딜레이는 그만큼 줄이고, 첫 반복은 tap 으로 같은 양을 늦춰서 모든 반복이 정확히 time 간격이 되게 함
  return {
    input: input,
    output: output,
    update: function (p) {
      const q = QUANTUM / ctx.sampleRate;
      const time = Math.max(2 * q, delaySeconds(p, getBpm()));
      delay.delayTime.value = time - q;
      tap.delayTime.value = q;
      feedback.gain.value = Math.max(0, Math.min(0.95, Number(p.feedback) || 0));
      tone.frequency.value = Math.max(200, Math.min(20000, Number(p.tone) || 6000));
      wet.gain.value = Math.max(0, Math.min(1, p.mix != null ? Number(p.mix) : 0.3));
    },
    dispose: function () { [input, output, delay, tone, feedback, tap, wet].forEach(drop); },
  };
}
```

3.5 Reverb

```
function createReverbFx(ctx) {
  const input = ctx.createGain();
  const output = ctx.createGain();
  const dry = ctx.createGain();
  const wet = ctx.createGain();
  const pre = ctx.createDelay(0.5);
  const conv = ctx.createConvolver();
  input.connect(dry); dry.connect(output);
  input.connect(pre); pre.connect(conv); conv.connect(wet); wet.connect(output);
  let key = '';
  return {
    input: input,
```

```
    output: output,
    update: function (p) {
      const decay = Math.max(0.1, Math.min(10, Number(p.decay) || 2));
      const damping = Math.max(0, Math.min(1, p.damping != null ? Number(p.damping) : 0.5));
      const next = decay + ':' + damping;
      if (next !== key) {
        key = next;
        const sr = ctx.sampleRate;
        const buf = ctx.createBuffer(2, Math.max(1, Math.floor(sr * decay)), sr);
        buf.copyToChannel(makeImpulse(sr, decay, damping, 0), 0);
        buf.copyToChannel(makeImpulse(sr, decay, damping, 1), 1);
        conv.buffer = buf;
      }
      pre.delayTime.value = Math.max(0, Math.min(0.5, Number(p.pre_delay) || 0));
      const mix = Math.max(0, Math.min(1, p.mix != null ? Number(p.mix) : 0.3));
      dry.gain.value = Math.cos(mix * Math.PI / 2);
      wet.gain.value = Math.sin(mix * Math.PI / 2) * 0.5;   // 컨볼버는 소리를 크게 만들어서 절반으로
    },
    dispose: function () { [input, output, dry, wet, pre, conv].forEach(drop); },
  };
}
```

3.6 만들기

```
export function createEffect(ctx, code, params, getBpm) {
  const make = FACTORIES[code];
  if (!make) return null;
  const fx = make(ctx, getBpm || function () { return 120; });
  fx.update(params || {});
  return fx;
}
```

4. 최신 문법으로 쓰면

```
// delaySeconds 를 구조 분해와 nullish 병합으로 쓴 모양
const DIVISION_BEATS = { '1/4': 1, '1/8': 0.5, '1/8d': 0.75, '1/16': 0.25 };
export const delaySeconds = ({ sync, division, time }, bpm) => {
  const sec = sync
    ? (DIVISION_BEATS[division] ?? 0.5) * 60 / (bpm > 0 ? bpm : 120)
    : Number(time) || 0.35;
  return Math.max(0.001, Math.min(MAX_DELAY_SECONDS, sec));
};
```

전통 문법 (실제 코드)	최신 문법
`DIVISION_BEATS[p.division] != null ? ... : 0.5`	`DIVISION_BEATS[division] ?? 0.5`
`p.mix != null ? Number(p.mix) : 0.3`	`p.mix ?? 0.3` (숫자 변환은 별도)
객체를 `p` 로 받아 `p.sync`	구조 분해 `{ sync, division, time }`

5. 왜 이렇게

선택	이유
이펙트를 같은 모양 `{ input, output, update, dispose }`로 통일했습니다	체인에서 이펙트 종류를 몰라도 `output`을 다음 `input`에 잇기만 하면 됩니다. 이펙트를 더하기 쉬워집니다.
노브가 바뀌면 `update`만 부릅니다	노드를 다시 만들면 소리가 끊깁니다. 값만 바꾸면 소리가 이어집니다.
리버브 임펄스를 고정 시드 난수로 만듭니다	같은 설정이면 늘 같은 잔향이 나야 합니다. 실시간과 WAV 출력, 회귀 시험이 같은 소리를 내게 하려는 것입니다.
임펄스는 `decay`와 `damping`이 바뀔 때만 다시 만듭니다	임펄스 생성은 계산이 많아서 믹스나 프리딜레이를 돌릴 때마다 만들지 않습니다.
리버브 믹스를 등출력 곡선으로 섞습니다	원음은 cos, 잔향은 sin이라 중간 값에서 전체 소리 크기가 일정합니다. 잔향은 컨볼버가 키우는 만큼 절반으로 줄입니다.
컴프레서와 리미터의 자동 메이크업을 덜어냅니다	브라우저의 컴프레서는 압축한 만큼 스스로 키워서, 그대로 쓰면 리미터가 천장을 넘고 메이크업 노브가 이중으로 걸립니다. 계산해서 같은 양을 줄입니다.
리미터는 근사치입니다	기본 컴프레서(레이시오 20, 니 0)로 흉내 낸 것이라 정확한 피크 리미터는 아닙니다. 이후 WASM으로 교체합니다 (`engine='WASM'`).
딜레이 고리에서 128샘플을 뺍니다	브라우저는 되먹임 고리를 한 바퀴 돌 때마다 128샘플을 더 걸리게 합니다. 고리의 시간을 줄이고 첫 반복을 따로 늦춰서 모든 반복이 정확한 간격이 됩니다.
Gain 이펙트의 입구를 스테레오로 고정했습니다	모노 소리가 `StereoPannerNode`를 지나면 가운데에서 -3dB가 됩니다. 이것을 막아 Gain을 -12dB로 두면 정확히 -12dB가 됩니다.
딜레이 시간을 템포에서 계산합니다	`sync`가 켜지면 박 길이(1/4, 1/8, 점8분, 1/16)와 BPM으로 초를 구합니다. BPM이 바뀌어도 맞도록 `EffectRack`가 매번 다시 계산합니다.

6. 시험

- `docs/handoff/s1-11a_tests/effects_unit.mjs` (17건)는 순수 함수를 확인합니다. 딜레이 시간은 120BPM의 1/4가 0.5초이고 1/8이 0.25초이며 1/8 점음표가 0.375초입니다. 임펄스는 길이가 맞고 같은 입력이면 같은 결과이며 좌우가 다르고 시간이 지나면 줄어듭니다. 댐핑이 크면 고음이 줄어듭니다. 자동 메이크업은 천장 -12dB, 레이시오 20에서 6.84dB입니다.
- `docs/handoff/s1-11a_tests/effects_offline.mjs` (16건)는 `OfflineAudioContext`로 실제 소리를 계산해서 확인합니다.

이펙트	확인한 내용
Gain	-12dB에서 크기가 약 1/4이고, +6dB에서 약 2배입니다.
EQ	저음 -24dB에서 100Hz 소리가 30% 아래로 줄고, 중음 +12dB에서 1kHz 소리가 약 4배이며, 모두 0dB이면 그대로입니다.
Compressor	임계값 -40dB, 20대 1에서 소리가 절반 아래로 줄고, 메이크업 +12dB에서 약 4배가 됩니다.
Limiter	천장 -12dB에서 입력 피크 0.5가 0.4 아래로 줄어듭니다.
Delay	원음이 그대로이고, 0.1초와 0.2초에 반복이 오며(되먹임 0.5), 120BPM의 1/8은 0.25초에 반복합니다.
Reverb	믹스 1이면 꼬리가 남고, 믹스 0이면 꼬리가 없으며, 같은 설정이면 렌더링 결과가 같습니다.

- 시험이 처음에 4건 실패했고, 모두 Web Audio의 동작 특성에서 비롯되었습니다. 자세한 내용은 문서 83의 10절을 봅니다.